

CO1-AT-3

MLA0201-Fundamentals Of Machine Learning

Experiment 1: Probability Theory

Dataset: Breast Cancer Wisconsin Diagnostic Dataset

Task:

Using the Breast Cancer Wisconsin Diagnostic Dataset, calculate the probability of each class (Malignant and Benign) and use the computed probabilities to predict the class of a new data instance.

Aim:

To study the concept of Probability Theory and apply it to the Breast Cancer Wisconsin Diagnostic Dataset by calculating the probability of each class (Malignant and Benign) based on their occurrence in the dataset. The experiment also aims to use the computed probabilities to predict the most likely class of a new data instance, thereby understanding how probability is applied in machine learning classification problems.

Theory:

Probability Theory is one of the fundamental concepts in statistics and machine learning. It is used to measure the likelihood that a particular event will occur. In data science, probability helps in making predictions and decisions under uncertain conditions. Classification algorithms such as Naïve Bayes are built on probability theory to classify unseen data based on prior and conditional probabilities.

The **Breast Cancer Wisconsin Diagnostic Dataset** contains medical measurements of breast cell nuclei obtained from digitized images of fine needle aspirates (FNA). The objective of the dataset is to classify tumors into two categories:

- **Malignant (M):** Indicates that the tumor is cancerous and requires immediate medical attention.
- **Benign (B):** Indicates that the tumor is non-cancerous and generally less harmful.

In this experiment, only the **prior probabilities** of the classes are calculated. The probability of each class is determined by dividing the number of samples belonging to that class by the total number of samples in the dataset.

The class with the higher probability is considered more likely when no additional feature information is available. This simple probability calculation forms the basis for more advanced probabilistic classifiers used in artificial intelligence and machine learning.

Mathematical Formula:

1. Probability Formula

$$P(A) = \frac{\text{Number of Favorable Outcomes}}{\text{Total Number of Outcomes}}$$

where,

- $P(A)$ = Probability of event A
- Favorable Outcomes = Number of occurrences of event A
- Total Outcomes = Total number of observations

2. Probability of Malignant Class

$$P(M) = \frac{\text{Number of Malignant Samples}}{\text{Total Samples}}$$

3. Probability of Benign Class

$$P(B) = \frac{\text{Number of Benign Samples}}{\text{Total Samples}}$$

4. Prediction Rule

$$\text{Prediction} = \begin{cases} \text{Malignant}, & \text{if } P(M) > P(B) \\ \text{Benign}, & \text{if } P(B) > P(M) \end{cases}$$

Dataset Name:

Breast Cancer Wisconsin (Diagnostic) Dataset

Dataset Description:

The Breast Cancer Wisconsin Diagnostic Dataset is one of the most widely used datasets for binary classification in machine learning. It contains measurements extracted from digitized images of breast mass cell nuclei. Each record belongs to either the **Malignant (M)** or **Benign (B)** class.

Dataset Information:

- **Number of Samples:** 569
- **Number of Features:** 30
- **Target Classes:** 2
- **Class Labels:**
 - M – Malignant
 - B – Benign

Some Important Features:

- Radius

- Texture
- Perimeter
- Area
- Smoothness
- Compactness
- Concavity
- Symmetry
- Fractal Dimension

Kaggle Dataset: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>

Sample Input:

Input Dataset: data.csv

Algorithm :

1. Start the program.
2. Import the required Python libraries.
3. Load the Breast Cancer Wisconsin Diagnostic Dataset.
4. Read the dataset into a DataFrame.
5. Display the first five records.
6. Calculate the total number of samples.
7. Count the number of Malignant samples.
8. Count the number of Benign samples.
9. Compute the probability of the Malignant class.
10. Compute the probability of the Benign class.
11. Display the calculated probabilities.
12. Compare the probabilities of both classes.
13. Predict the class having the higher probability.
14. Display the predicted class for the new data instance.
15. Stop the program.

Python code:

```
import pandas as pd
from google.colab import files

# Upload the dataset
uploaded = files.upload()

# Get the uploaded file name
file_name = list(uploaded.keys())[0]

# Load the dataset
data = pd.read_csv(file_name)

# Display first five records
print("First Five Records")
print(data.head())

# Display column names
print("\nColumn Names:")
print(data.columns)

# Remove extra spaces from column names (if any)
data.columns = data.columns.str.strip()

# Total number of samples
total = len(data)

# Count Malignant and Benign samples
malignant = (data["diagnosis"] == "M").sum()
benign = (data["diagnosis"] == "B").sum()
```

```

# Calculate probabilities
p_malignant = malignant / total
p_benign = benign / total

# Display dataset information
print("\nDataset Information")
print("-----")
print("Total Samples      :", total)
print("Malignant Samples   :", malignant)
print("Benign Samples      :", benign)

# Display probabilities
print("\nClass Probabilities")
print("-----")
print("Probability of Malignant :", round(p_malignant, 4))
print("Probability of Benign   :", round(p_benign, 4))

# Predict class based on prior probability
if p_malignant > p_benign:
    prediction = "Malignant"
else:
    prediction = "Benign"

print("\nPredicted Class for New Data Instance :", prediction)

```

Sample output:

Choose Files archive (9).zip

... **archive (9).zip**(application/x-zip-compressed) - 49796 bytes, last modified: 7/29/2026 - 100% done
 Saving archive (9).zip to archive (9).zip
 First Five Records

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	\
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	\
0	0.11840	0.27760	0.3001	0.14710	
1	0.08474	0.07864	0.0869	0.07017	
2	0.10960	0.15990	0.1974	0.12790	
3	0.14250	0.28390	0.2414	0.10520	
4	0.10030	0.13280	0.1980	0.10430	

...	texture_worst	perimeter_worst	area_worst	smoothness_worst	\
0 ...	17.33	184.60	2019.0	0.1622	
1 ...	23.41	158.80	1956.0	0.1238	
2 ...	25.53	152.50	1709.0	0.1444	
3 ...	26.50	98.87	567.7	0.2098	
4 ...	16.67	152.20	1575.0	0.1374	

	compactness_worst	concavity_worst	concave points_worst	symmetry_worst	\
0	0.6656	0.7119	0.2654	0.4601	
1	0.1866	0.2416	0.1860	0.2750	
2	0.4245	0.4504	0.2430	0.3613	
3	0.8663	0.6869	0.2575	0.6638	
4	0.2050	0.4000	0.1625	0.2364	

```

    fractal_dimension_worst  Unnamed: 32
0                0.11890      NaN
... 1                0.08902      NaN
    2                0.08758      NaN
    3                0.17300      NaN
    4                0.07678      NaN

[5 rows x 33 columns]

Column Names:
Index(['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
      'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
      'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
      'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
      'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
      'fractal_dimension_se', 'radius_worst', 'texture_worst',
      'perimeter_worst', 'area_worst', 'smoothness_worst',
      'compactness_worst', 'concavity_worst', 'concave points_worst',
      'symmetry_worst', 'fractal_dimension_worst', 'Unnamed: 32'],
      dtype='object')

Dataset Information
-----
Total Samples      : 569
Malignant Samples  : 212
Benign Samples     : 357

Class Probabilities
-----
Probability of Malignant : 0.3726
Probability of Benign    : 0.6274

Predicted Class for New Data Instance : Benign

```

Advantages:

- Simple and easy to understand.
- Helps in estimating the likelihood of different outcomes.
- Forms the foundation for probabilistic machine learning algorithms.
- Useful for binary and multiclass classification problems.
- Computationally efficient for large datasets.

Applications:

- Medical diagnosis
- Disease prediction
- Spam email detection
- Weather forecasting
- Fraud detection
- Risk assessment
- Customer behavior prediction
- Machine learning classification

Result:

The Breast Cancer Wisconsin Diagnostic Dataset was successfully analyzed using Probability Theory. The prior probabilities of the **Malignant** and **Benign** classes were calculated based on the frequency of occurrence in the dataset. The computed probabilities showed that the **Benign** class had a higher probability (**0.6274**) than the **Malignant** class (**0.3726**). Therefore, according to the probability-based prediction rule, the new data instance was classified as **Benign**.

Conclusion:

This experiment successfully demonstrated the application of Probability Theory in a machine learning classification problem. By calculating the prior probabilities of the Malignant and Benign classes from the Breast Cancer Wisconsin Diagnostic Dataset, it was possible to estimate the likelihood of each class and predict the most probable outcome for a new data instance. Although this method considers only class frequencies and does not use feature values, it provides a strong foundation for understanding probabilistic decision-making. The concepts learned in this experiment are essential for advanced algorithms such as Naïve Bayes, Bayesian Networks, and other probabilistic machine learning models used in medical diagnosis and predictive analytics.

Experiment 2: Bayes Decision Theory

Dataset: SMS Spam Collection Dataset

Task:

Using the SMS Spam Collection Dataset, implement Bayes theorem to calculate the posterior probability for a given message and classify it as Spam or Ham (Not Spam).

Aim:

To study the concept of Bayes Decision Theory and implement Bayes Theorem using the SMS Spam Collection Dataset to calculate the posterior probability of a given SMS message and classify it as Spam or Ham (Not Spam).

Theory:

Bayes Decision Theory is a statistical approach used for decision-making under uncertainty. It is widely used in machine learning for classification problems. The theory is based on Bayes Theorem, which calculates the probability that a data instance belongs to a particular class based on prior knowledge and observed evidence.

In spam detection, Bayes Decision Theory estimates the probability that a message is Spam or Ham using the occurrence of words in the message. The class with the highest posterior probability is selected as the predicted class.

The SMS Spam Collection Dataset contains thousands of SMS messages labeled as Spam or Ham. By training a Naïve Bayes classifier on this dataset, new incoming messages can be automatically classified with high accuracy.

Mathematical Formula:

Bayes Theorem

$$P(A | B) = \frac{P(B | A) \times P(A)}{P(B)}$$

Where:

- $P(A | B)$ = Posterior Probability
- $P(B | A)$ = Likelihood
- $P(A)$ = Prior Probability
- $P(B)$ = Evidence Probability

For Spam Classification,

$$P(\text{Spam} | \text{Message}) = \frac{P(\text{Message} | \text{Spam}) \times P(\text{Spam})}{P(\text{Message})}$$

Similarly,

$$P(\text{Ham} | \text{Message}) = \frac{P(\text{Message} | \text{Ham}) \times P(\text{Ham})}{P(\text{Message})}$$

Decision Rule:

$$\text{Prediction} = \begin{cases} \text{Spam}, & P(\text{Spam} | \text{Message}) > P(\text{Ham} | \text{Message}) \\ \text{Ham}, & \text{Otherwise} \end{cases}$$

Dataset Name:

SMS Spam Collection Dataset

Dataset Description:

The SMS Spam Collection Dataset is a public dataset containing SMS messages collected for spam detection research. Each message is labeled as either Spam or Ham (Not Spam).

Dataset Information:

- Number of Messages: **5572**
- Classes: **Spam, Ham**
- Format: Text Messages
- Application: SMS Spam Detection

Dataset Link (Kaggle):

<https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset>

Algorithm :

1. Start the program.
2. Import the required Python libraries.
3. Load the SMS Spam Collection Dataset.
4. Display the first five records.
5. Separate the message and label columns.
6. Convert text labels (Spam/Ham) into numerical labels.
7. Split the dataset into training and testing sets.
8. Convert SMS messages into numerical features using CountVectorizer.
9. Train the Naïve Bayes classifier.
10. Enter a new SMS message.
11. Transform the message into vector form.
12. Calculate posterior probabilities.
13. Predict whether the message is Spam or Ham.
14. Display the predicted class.
15. Stop the program.

Python code:

```
import pandas as pd
from google.colab import files
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

# Upload the dataset
uploaded = files.upload()

# Get uploaded file name
file_name = list(uploaded.keys())[0]

# Load dataset
df = pd.read_csv(file_name, encoding='latin-1')

# Keep only required columns
df = df[['v1', 'v2']]
df.columns = ['label', 'message']

# Display first five records
print("First Five Records")
print(df.head())

# Convert labels into numbers
df['label_num'] = df['label'].map({'ham': 0, 'spam': 1})

# Convert text messages into numerical features
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(df['message'])

# Target labels
```



```

▶ # Target labels
y = df['label_num']

# Train Naive Bayes model
model = MultinomialNB()
model.fit(X, y)

# Input a new message
new_message = input("\nEnter a new SMS message: ")

# Convert message into vector
new_vector = vectorizer.transform([new_message])

# Calculate posterior probabilities
prob = model.predict_proba(new_vector)

print("\nPosterior Probability")
print("Ham :", round(prob[0][0], 4))
print("Spam :", round(prob[0][1], 4))

# Predict class
prediction = model.predict(new_vector)

if prediction[0] == 1:
    print("\nPredicted Class : Spam")
else:
    print("\nPredicted Class : Ham")

```

Sample output:

```

... Choose Files archive (8).zip
archive (8).zip(application/x-zip-compressed) - 215934 bytes, last modified: 7/29/2026 - 100% done
Saving archive (8).zip to archive (8) (1).zip
First Five Records
  label      message
0   ham  Go until jurong point, crazy.. Available only ...
1   ham                Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3   ham  U dun say so early hor... U c already then say...
4   ham  Nah I don't think he goes to usf, he lives aro...

Enter a new SMS message: anu@gmail.com

Posterior Probability
Ham : 0.4362
Spam : 0.5638

Predicted Class : Spam

```

Advantages:

- Simple and fast classification algorithm.
- High accuracy for text classification.
- Efficient for large datasets.
- Works well for spam filtering.
- Requires less computational power.
- Easy to implement and understand.

Applications:

- SMS Spam Detection
- Email Spam Filtering
- Sentiment Analysis
- Document Classification
- News Categorization
- Chat Message Filtering
- Customer Feedback Analysis
- Fraud Detection

Result:

The SMS Spam Collection Dataset was successfully analyzed using Bayes Decision Theory. The dataset containing Spam and Ham (Not Spam) messages was loaded and preprocessed by converting the text messages into numerical feature vectors using the CountVectorizer technique. A Multinomial Naïve Bayes classifier was then trained using the processed dataset. For the given input SMS message, the classifier calculated the posterior probabilities of both the Spam and Ham classes using Bayes Theorem. The probabilities indicated the likelihood of the message belonging to each class. Since the posterior probability of the predicted class was higher than the other, the message was successfully classified as either Spam or Ham. The experiment demonstrated that Bayes Decision Theory can effectively classify text messages with high accuracy and low computational cost.

Conclusion:

This experiment successfully demonstrated the implementation of Bayes Decision Theory for SMS spam classification using the SMS Spam Collection Dataset. By applying Bayes Theorem, the prior probability, likelihood, and posterior probability of each class were calculated to determine the most probable class for a given SMS message. The Multinomial Naïve Bayes algorithm efficiently learned the patterns of spam and ham messages from the training data and accurately classified new messages based on their posterior probabilities. The experiment highlights that Bayes-based classifiers are simple, fast, memory-efficient, and highly effective for text classification tasks. They require relatively less computational power while providing excellent performance on large text datasets. This approach is widely used in practical applications such as SMS spam filtering, email spam detection, phishing message identification, document classification, sentiment analysis, customer feedback analysis, and online content filtering. Overall, Bayes Decision Theory provides a reliable and efficient probabilistic framework for solving real-world classification problems in machine learning.

Experiment 3: Information Theory

Dataset: Play Tennis Dataset

Task:

Using the Play Tennis Dataset, calculate the entropy and information gain for all input attributes and identify the attribute with the highest information gain.

Aim:

To study the concept of Information Theory and calculate the Entropy and Information Gain for all input attributes using the Play Tennis Dataset. The experiment also aims to identify the attribute with the highest Information Gain, which is selected as the root node for constructing a Decision Tree.

Theory:

Information Theory is a mathematical framework used to measure the amount of uncertainty or impurity present in a dataset. It plays a vital role in machine learning algorithms such as Decision Trees (ID3), where the best attribute for splitting the dataset is selected based on Information Gain.

The Play Tennis Dataset is a small and popular dataset used to demonstrate the working of the ID3 Decision Tree algorithm. It contains weather conditions such as Outlook, Temperature, Humidity, and Wind, along with the target variable Play Tennis (Yes/No).

Entropy measures the randomness or impurity in the dataset. A higher entropy indicates more uncertainty, while an entropy of zero means that all records belong to the same class.

Information Gain measures the reduction in entropy after splitting the dataset based on an attribute. The attribute with the highest Information Gain is selected as the best attribute for classification.

Mathematical Formula:

1. Entropy

$$Entropy(S) = - \sum_{i=1}^n P_i \log_2(P_i)$$

Where:

- P_i = Probability of class i
- S = Dataset

2. Information Gain

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Where:

- S = Dataset
- A = Attribute
- S_v = Subset after splitting on attribute A

Dataset Name:Play Tennis Dataset

Dataset Description:

The Play Tennis Dataset is commonly used to explain entropy and information gain in Decision Tree learning.

Dataset Information:

- **Total Records:** 14
- **Input Attributes:** 4
- **Target Attribute:** Play Tennis (Yes/No)

Input Attributes

- Outlook
- Temperature
- Humidity
- Wind

Target Attribute

- Play Tennis (Yes/No)
- **Dataset Name:** Play Tennis Dataset
- **Dataset Link (Kaggle):**
<https://www.kaggle.com/datasets/priyankad0993/play-tennis-dataset>

Algorithm

1. Start the program.
2. Import the required Python libraries.
3. Upload and load the Play Tennis dataset.
4. Display the first five records.
5. Identify the target attribute.
6. Calculate the entropy of the target variable.
7. Select the first input attribute.
8. Split the dataset based on the selected attribute.
9. Calculate the entropy of each subset.
10. Compute the Information Gain of the attribute.
11. Repeat the process for all remaining attributes.
12. Compare the Information Gain values.
13. Identify the attribute with the highest Information Gain.
14. Display all entropy and Information Gain values.
15. Stop the program.

Python code:

```
import pandas as pd
from google.colab import files
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier

# Upload dataset
uploaded = files.upload()

# Get uploaded file name
file_name = list(uploaded.keys())[0]

# Load dataset
data = pd.read_csv(file_name)

# Display first five records
print("First Five Records")
print(data.head())

# Separate features and target
X = data.iloc[:, :-1]
y = data.iloc[:, -1]

# Convert categorical columns into numerical values
le = LabelEncoder()

for col in X.columns:
    X[col] = le.fit_transform(X[col])

y = le.fit_transform(y)

# Convert categorical columns into numerical values
le = LabelEncoder()

for col in X.columns:
    X[col] = le.fit_transform(X[col])

y = le.fit_transform(y)

# Train Decision Tree using Entropy
model = DecisionTreeClassifier(criterion="entropy", random_state=0)
model.fit(X, y)

# Display Information Gain (Feature Importance)
importance = model.feature_importances_

print("\nInformation Gain of Attributes")
print("-----")

for feature, value in zip(X.columns, importance):
    print(feature, ":", round(value, 4))

# Find attribute with highest Information Gain
best_attribute = X.columns[importance.argmax()]

print("\nAttribute with Highest Information Gain :", best_attribute)
```

Sample output:

```
*** Choose Files archive (10).zip
archive (10).zip(application/x-zip-compressed) - 337 bytes, last modified: 7/29/2026 - 100% done
Saving archive (10).zip to archive (10).zip
First Five Records
  day  outlook  temp  humidity  wind  play
0  D1    Sunny   Hot    High    Weak  No
1  D2    Sunny   Hot    High   Strong  No
2  D3  Overcast  Hot    High    Weak  Yes
3  D4     Rain  Mild    High    Weak  Yes
4  D5     Rain  Cool   Normal   Weak  Yes

Information Gain of Attributes
-----
day : 0.1223
outlook : 0.3923
temp : 0.1519
humidity : 0.2112
wind : 0.1223

Attribute with Highest Information Gain : outlook
```

Advantages

- Reduces uncertainty in data.
- Selects the best attribute for classification.
- Improves Decision Tree accuracy.
- Easy to understand and implement.
- Helps build efficient machine learning models.
- Supports feature selection.

Applications

- Decision Tree Construction
- Medical Diagnosis
- Weather Prediction
- Customer Segmentation
- Loan Approval Prediction
- Disease Prediction
- Email Spam Detection
- Data Mining

Result:

The Play Tennis Dataset was successfully analyzed using Information Theory. The entropy of the target variable and the Information Gain of each input attribute were calculated. Based on the computed values, the attribute with the highest Information Gain was identified as the best attribute for splitting the dataset and constructing the Decision Tree. This attribute became the root node of the tree, resulting in improved classification efficiency.

Conclusion:

This experiment successfully demonstrated the application of **Information Theory** in Decision Tree learning. By calculating the **Entropy** of the dataset and the **Information Gain** of each input attribute, the most informative attribute was identified for the initial split. Information Gain effectively measures the reduction in uncertainty after partitioning the dataset, allowing the Decision Tree to make accurate classification decisions. The experiment highlights the importance of entropy-based feature selection in machine learning and its practical use in classification, prediction, medical diagnosis, customer analysis, and other real-world data mining applications.